

Anexo técnico

vineGAPdetect

Índice

1	Frontend	2
1.1	Rutas React	2
1.2	Componentes y módulos documentados	2
2	Backend	6
2.1	Arranque y composición	6
2.2	Endpoints FastAPI	6
2.3	Servicios internos	10
3	Módulo ML	14
3.1	Pipeline experimental y librería de explicabilidad	14

Capítulo 1

Frontend

1.1 Rutas React

/	LoginPage
/dashboard	ProtectedDashboard
/history	HistoryPage
/admin/users	AdminUsersPage
/admin/users/:userId	AdminUsersPage
/admin/users/new	AdminUserCreatePage

1.2 Componentes y módulos documentados

App (function)

Fichero: frontend/src/app/App.tsx

Compone autenticacion, estado del dashboard y enrutado principal de la aplicacion web. El arbol raiz garantiza que todas las pantallas accedan a la sesion activa y al estado compartido del analisis sin duplicar logica entre rutas.

AuthProvider (function)

Fichero: frontend/src/app/auth/AuthContext.tsx

Proporciona token de sesion, usuario autenticado y acciones de entrada y salida. Al arrancar intenta recuperar el token almacenado, valida la sesion contra el backend y sincroniza el cliente API con la cabecera Authorization.

useAuth (function)

Fichero: frontend/src/app/auth/AuthContext.tsx

Devuelve el contexto de autenticacion actual y exige el uso dentro del proveedor.

createInferenceJob (function)

Fichero: frontend/src/app/api/client.ts

Envia una imagen al backend y crea un job asincrono de deteccion de marras. Empaqueta archivo y parametros agronomicos en multipart/form-data para que FastAPI pueda procesar imagenes grandes sin bloquear la interfaz.

generatePreview (function)

Fichero: frontend/src/app/api/client.ts

Solicita la previsualizacion y metadatos de una imagen antes de ejecutar la inferencia.

createXaiJob (function)

Fichero: frontend/src/app/api/client.ts

Crea un job Eigen-CAM sobre la imagen cargada y una region opcional de interes. Permite fijar capa objetivo, umbral, tamano de parche y coordenadas o bbox de foco para explicar una deteccion concreta.

createHistoryXaiJob (function)

Fichero: frontend/src/app/api/client.ts

Regenera XAI desde una entrada de historial que conserva el artefacto fuente.

getJob (function)

Fichero: frontend/src/app/api/client.ts

Consulta estado y resultado de un job validando previamente su identificador UUID.

getHistory (function)

Fichero: frontend/src/app/api/client.ts

Recupera el listado de analisis historicos visibles para el usuario.

getHistoryItem (function)

Fichero: frontend/src/app/api/client.ts

Recupera el detalle de un analisis historico para restaurarlo en el dashboard.

exportDetectionsGpkg (function)

Fichero: frontend/src/app/api/client.ts

Descarga un GeoPackage construido a partir de detecciones y transformacion geoespacial. Solo se invoca cuando la inferencia conserva CRS y matriz afin del GeoTIFF, porque esos datos son necesarios para transformar pixeles a coordenadas GIS.

DashboardStateProvider (function)

Fichero: frontend/src/app/hooks/DashboardStateContext.tsx

Comparte el estado del flujo de analisis entre paneles y lo reinicia al cerrar sesion.

useDashboardStateContext (function)

Fichero: frontend/src/app/hooks/DashboardStateContext.tsx

Devuelve el contexto de estado usado por la pantalla principal de analisis.

getDefaultXaiLayers (function)

Fichero: frontend/src/app/hooks/useDashboardState.ts

Devuelve la capa por defecto usada para Eigen-CAM en el modelo entrenado.

useDashboardState (function)

Fichero: frontend/src/app/hooks/useDashboardState.ts

Centraliza el estado del dashboard: imagen, metadatos, inferencia, XAI, historial y errores. Coordina subida de archivo, generacion de previsualizacion, creacion de jobs, polling de resultados, restauracion desde historial, seleccion de deteccion para XAI y limpieza de URLs temporales del navegador.

useJobPolling (function)

Fichero: frontend/src/app/hooks/useJobPolling.ts

Consulta periodicamente un job del backend hasta completarse o fallar y expone su progreso.

Dashboard (function)

Fichero: frontend/src/app/components/Dashboard.tsx

Orquesta el flujo principal: carga de imagen, inferencia, XAI, filtros y exportaciones. Conecta el estado global con los paneles visuales, recalcula marras segun el ancho de cepa elegido por el usuario y habilita exportacion PDF/GPKG cuando existen detecciones y metadatos suficientes.

InferenceCanvasHandle (interface)

Fichero: frontend/src/app/components/InferenceCanvas.tsx

API imperativa usada por la barra de herramientas para controlar zoom y encuadre.

InferenceCanvas (const)

Fichero: frontend/src/app/components/InferenceCanvas.tsx

Visualiza la imagen analizada, superpone XAI y dibuja detecciones OBB con OpenLayers. Crea una proyeccion en coordenadas de pixel, representa la imagen como capa estatica, dibuja poligonos o cajas de deteccion y comunica escala y numero de entidades visibles al resto del dashboard.

HistoryPage (function)

Fichero: frontend/src/app/components/HistoryPage.tsx

Muestra analisis guardados y permite restaurarlos en el dashboard con sus metadatos. La pantalla recupera resúmenes historicos, ordena resultados, borra registros y reconstruye el estado necesario para inspeccionar detecciones sin repetir la inferencia.

ProtectedDashboard (function)

Fichero: frontend/src/app/components/ProtectedDashboard.tsx

Protege la ruta del dashboard y redirige usuarios anonimos al inicio de sesion.

AdminUsersPage (function)

Fichero: frontend/src/app/components/admin/UsersPage.tsx

Pantalla administrativa para listar, editar, desactivar, borrar y reiniciar contraseñas.

AdminUserCreatePage (function)

Fichero: frontend/src/app/components/admin/UserCreatePage.tsx

Formulario administrativo para crear usuarios con rol y estado inicial.

PdfReportData (interface)

Fichero: frontend/src/app/utils/exportPdf.ts

Datos necesarios para construir el informe PDF local del analisis mostrado.

generatePdfReport (function)

Fichero: frontend/src/app/utils/exportPdf.ts

Genera en el navegador un PDF con metadatos, metricas e imagen anotada del analisis. El informe se construye localmente con jsPDF para no enviar de nuevo datos al backend y usa una imagen reducida con detecciones pintadas segun confianza.

Capítulo 2

Backend

2.1 Arranque y composición

backend/app/main.py (module)

Fichero: backend/app/main.py

Punto de entrada de la API FastAPI de vineGAPdetect. El modulo compone la aplicacion backend: inicializa la base de datos SQLite, crea los servicios de usuarios, autenticacion, gestion de jobs, inferencia y explicabilidad, configura CORS para el frontend y registra todas las rutas bajo el prefijo de API. El ciclo de vida inicia el worker de trabajos al arrancar y lo detiene al cerrar la aplicacion.

2.2 Endpoints FastAPI

POST /api/v1/auth/login

Handler login

Acceso publico

Respuesta SessionResponse

Autentica un usuario y crea una sesion con token bearer.

POST /api/v1/auth/logout

Handler logout

Acceso usuario autenticado

Respuesta MessageResponse

Invalida la sesion activa y confirma el cierre al usuario.

GET /api/v1/auth/me

Handler me

Acceso usuario autenticado

Respuesta UserPublic

Devuelve los datos publicos del usuario asociado al token actual.

POST /api/v1/export/detections-gpkg

Handler export_detections_gpkg

Acceso usuario autenticado

Respuesta -

Genera un GeoPackage con las detecciones georreferenciadas del analisis. Recibe detecciones filtradas, CRS y transformacion afin del GeoTIFF original, convierte poligonos OBB de pixeles a coordenadas reales y devuelve un fichero GPKG descargable para uso en software GIS.

GET /api/v1/health

Handler health

Acceso publico

Respuesta HealthResponse

Informa del estado de la API, el dispositivo configurado y la disponibilidad del modelo.

GET /api/v1/history

Handler get_history

Acceso usuario autenticado

Respuesta list[AnalysisHistoryItem]

Lista analisis historicos visibles para el usuario autenticado. Los administradores consultan todos los registros y los operarios solo los suyos. La respuesta contiene resúmenes, parametros usados y metadatos de imagen suficientes para navegar y filtrar el historico.

GET /api/v1/history/quota

Handler get_user_quota

Acceso usuario autenticado

Respuesta QuotaResponse

Devuelve el espacio usado y el limite de almacenamiento del usuario. Calcula la cuota a partir de las parcelas guardadas en el historico y permite al frontend bloquear nuevas conservaciones antes de superar el maximo configurado para cada cuenta.

DELETE /api/v1/history/{history_id}

Handler delete_history_item

Acceso usuario autenticado

Respuesta -

Borra un analisis historico y su artefacto fuente si existe.

GET /api/v1/history/{history__id}**Handler** get_history_item**Acceso** usuario autenticado**Respuesta** AnalysisHistoryDetail

Recupera detalle, previsualizacion, resultado y XAI almacenados de un analisis. Devuelve la informacion necesaria para restaurar el dashboard: imagen previsualizada, JSON de detecciones, parametros de ejecucion, metadatos geoespaciales y mapa XAI precalculado si existe.

POST /api/v1/history/{history__id}/save**Handler** save_history_item**Acceso** usuario autenticado**Respuesta** SaveParcelResponse

Marca una inferencia historica como parcela guardada del usuario. Verifica permisos, evita duplicados por hash o metadatos de fichero, comprueba la cuota disponible y conserva el registro para que aparezca en Mis parcelas sin duplicar el artefacto fuente.

POST /api/v1/history/{history__id}/xai**Handler** create_history_xai_job**Acceso** usuario autenticado**Respuesta** JobAcceptedResponse

Crea un trabajo XAI reutilizando la imagen original guardada en el historial. Recupera el artefacto fuente persistido, selecciona la deteccion indicada o la mejor segun umbral, calcula su centro y lanza un job Eigen-CAM enfocado sobre esa marra sin repetir la inferencia completa.

POST /api/v1/jobs/inference**Handler** create_inference_job**Acceso** operador/administrador**Respuesta** JobAcceptedResponse

Registra un trabajo asincrono de inferencia YOLO sobre la imagen subida. Valida tamano y disponibilidad del modelo, conserva parametros de corte por teselas, umbral de confianza, solape y ancho tipico de cepa, y delega la ejecucion en JobManager para que el frontend consulte progreso sin bloquear.

POST /api/v1/jobs/xai**Handler** create_xai_job**Acceso** operador/administrador**Respuesta** JobAcceptedResponse

Registra un trabajo asincrono de explicabilidad Eigen-CAM. Acepta una imagen y parametros de foco, valida que el metodo soportado sea Eigen-CAM y prepara capas objetivo, umbral, tamano de parche y bbox opcional antes de delegar el calculo al servicio XAI.

DELETE /api/v1/jobs/{job_id}**Handler** delete__job**Acceso** usuario autenticado**Respuesta** DeleteJobResponse

Elimina del gestor un trabajo que ya no esta en ejecucion.

GET /api/v1/jobs/{job_id}**Handler** get__job__status**Acceso** usuario autenticado**Respuesta** JobStatusResponse

Consulta estado, progreso, error y resultado de un trabajo por identificador.

GET /api/v1/model/info**Handler** model__info**Acceso** usuario autenticado**Respuesta** ModelInfoResponse

Expone informacion operativa del modelo cargado y parametros por defecto.

POST /api/v1/preview**Handler** generate__preview**Acceso** usuario autenticado**Respuesta** PreviewImageResponse

Carga una imagen o GeoTIFF y devuelve previsualizacion junto con metadatos geoespaciales. Esta ruta permite al frontend mostrar la parcela antes de ejecutar el modelo. Para GeoTIFF conserva resolucion, CRS, transformacion afin, area, centro y fecha de adquisicion cuando estan disponibles.

GET /api/v1/users**Handler** list__users**Acceso** administrador**Respuesta** list[UserPublic]

Lista todos los usuarios registrados para la vista de administracion.

POST /api/v1/users**Handler** create__user**Acceso** administrador**Respuesta** UserPublic

Crea un usuario con rol operativo o administrativo.

DELETE /api/v1/users/{user_id}

Handler delete__user

Acceso administrador

Respuesta MessageResponse

Elimina un usuario impidiendo que un administrador se borre a si mismo.

GET /api/v1/users/{user_id}

Handler get__user

Acceso administrador

Respuesta UserPublic

Obtiene el detalle publico de un usuario gestionado.

PATCH /api/v1/users/{user_id}

Handler update__user

Acceso administrador

Respuesta UserPublic

Actualiza datos administrativos de un usuario existente.

POST /api/v1/users/{user_id}/reset-password

Handler reset__password

Acceso administrador

Respuesta MessageResponse

Restablece la contraseña de un usuario desde la administracion.

2.3 Servicios internos

Settings (class)

Fichero: backend/app/core/config.py

Configuracion central de API, rutas de datos, modelo, trabajos y valores por defecto. Reune en una unica clase los parametros que condicionan el despliegue: prefijo de API, origen permitido para CORS, ubicacion de SQLite, carpeta de artefactos historicos, ruta del modelo YOLO, dispositivo de inferencia y valores por defecto usados en inferencia y explicabilidad.

AuthService (class)

Fichero: backend/app/services/auth_service.py

Gestiona login, logout y validacion de tokens persistidos en base de datos.

backend/app/services/gpkg_export.py (module)

Fichero: backend/app/services/gpkg_export.py

Construcción de GeoPackage (.gpkg) con las detecciones georreferenciadas. Las detecciones llegan con 'obb_polygon' en coordenadas de píxel del TIFF original. Aplicamos la transformación afin de rasterio para llevar cada vértice a coordenadas del CRS del ortomosaico, y serializamos como capa de polígonos en un único GPKG. Este modulo es la frontera entre la salida del modelo, que trabaja en pixeles, y el resultado GIS que puede abrirse en herramientas externas como QGIS.

build_detections_gpkg (function)

Fichero: backend/app/services/gpkg_export.py

Construye un GeoPackage en memoria a partir de detecciones OBB y metadatos CRS. Filtra detecciones por confianza, transforma vertices con la matriz afin del raster original, conserva atributos relevantes de cada marra y devuelve el fichero como bytes para descarga HTTP.

LoadedImage (class)

Fichero: backend/app/services/image_service.py

Imagen RGB normalizada junto con metadatos raster y geoespaciales extraídos. Es el contrato interno que permite tratar de forma homogénea imágenes convencionales y ortomosaicos GeoTIFF. Incluye dimensiones, resolución, unidad, CRS, transformación afin, área aproximada, centro geográfico y fecha de adquisición cuando esa información está disponible en el raster.

load_geotiff_from_bytes (function)

Fichero: backend/app/services/image_service.py

Lee un GeoTIFF desde memoria y extrae imagen RGB, resolución, CRS y metadatos de parcela. Normaliza bandas raster a 8 bits para visualización e inferencia, conserva la transformación pixel-CRS necesaria para exportar resultados y calcula metadatos agronómicos básicos como área aproximada y centro geográfico.

extract_geotiff_patch_from_bytes (function)

Fichero: backend/app/services/image_service.py

Extrae una ventana RGB de tamaño fijo desde un GeoTIFF sin cargar todo el raster. Se usa en XAI para trabajar solo sobre el parche analizado, reduciendo memoria y manteniendo coherencia con imágenes geoespaciales grandes.

load_regular_image_from_bytes (function)

Fichero: backend/app/services/image_service.py

Carga una imagen convencional con Pillow y asigna resolución de píxel no georreferenciada.

load_image_from_upload (function)

Fichero: backend/app/services/image_service.py

Selecciona el cargador adecuado según extensión GeoTIFF o formato de imagen estándar.

encode_png_base64 (function)

Fichero: backend/app/services/image_service.py

Codifica una imagen RGB como PNG base64 para respuestas JSON.

encode__preview__jpeg (function)

Fichero: backend/app/services/image_service.py

Genera una previsualización JPEG reducida y codificada en base64.

resize_for_preview (function)

Fichero: backend/app/services/image_service.py

Reduce una imagen manteniendo proporción para uso interactivo en el frontend.

calculate__missing__vines (function)

Fichero: backend/app/services/inference_service.py

Estima marras dividiendo la longitud física de la detección por el ancho típico de cepa.

InferenceService (class)

Fichero: backend/app/services/inference_service.py

Ejecuta detección por teselas, calcula métricas y persiste el análisis en historial. Recibe la imagen subida, normaliza su lectura con ImageService, divide la imagen en ventanas mediante SAHI y aplica el modelo YOLO para detectar marras. Convierte cada predicción a una geometría OBB o bbox, calcula el número estimado de cepas ausentes según resolución y ancho típico, construye resúmenes agregados y guarda resultado, previsualización y artefacto fuente para poder restaurar el análisis posteriormente.

JobRecord (class)

Fichero: backend/app/services/job_manager.py

Estado serializable de un trabajo asíncrono ejecutado por la API.

JobManager (class)

Fichero: backend/app/services/job_manager.py

Ejecuta trabajos en segundo plano y conserva progreso, errores y resultados temporales. Desacopla las operaciones lentas de la petición HTTP: las rutas crean un job, el worker procesa inferencia o XAI, y el frontend consulta el estado mediante polling. Incluye limpieza periódica para no mantener resultados temporales indefinidamente en memoria.

ModelRegistry (class)

Fichero: backend/app/services/model_registry.py

Carga y reutiliza modelos YOLO, adaptador SAHI y módulos CAM según la configuración. La carga es perezosa para evitar inicializar modelos pesados hasta que una ruta los necesita. Expone un modelo Ultralytics directo para XAI, un modelo SAHI para inferencia por teselas y comprueba si la librería local yolo_cam está disponible para generar mapas Eigen-CAM.

StoredUser (class)

Fichero: backend/app/services/user_service.py

Representación interna de usuario con hash de contraseña.

UserService (class)

Fichero: backend/app/services/user_service.py

Administra usuarios, roles, autenticación de credenciales y migración inicial.

XAIService (class)

Fichero: backend/app/services/xai_service.py

Genera explicabilidad Eigen-CAM sobre una imagen completa o una deteccion concreta. Selecciona una region de interes a partir del centro indicado, una bbox historica o la deteccion de mayor confianza. Extrae un parche consistente tambien para GeoTIFF, ejecuta el modelo YOLO y compone imagen original, anotacion y mapa de activacion para que el frontend pueda mostrar la explicabilidad sin recalcularla en el navegador.

Capítulo 3

Módulo ML

3.1 Pipeline experimental y librería de explicabilidad

ml/configs/dataset_config.py (module)

Fichero: ml/configs/dataset_config.py

Configuración centralizada del pipeline ML de vineGAPdetect. Pensada para ser portable: - Usa rutas relativas al directorio 'ml/' por defecto. - Permite sobrescribir las rutas principales mediante variables de entorno.

ml/scripts/generar_parches.py (module)

Fichero: ml/scripts/generar_parches.py

Script de creación de parches y etiquetas YOLOv11-OBB. Este script permite generar un dataset para entrenar un modelo que detecte una única clase de interés y filtrar artefactos en los bordes.

PatcherConfig (class)

Fichero: ml/scripts/generar_parches.py

Parametros reproducibles para convertir rasters y vectores en dataset YOLO-OBB. Define origen de datos, destino, tamaño de parche, solape, particiones train/validation/test, clase objetivo y reglas de filtrado. Es la unidad de configuración que permite repetir la generación del dataset experimental.

PatcherStats (class)

Fichero: ml/scripts/generar_parches.py

Acumula contadores y tiempos del proceso de generación de parches.

YoloPatcher (class)

Fichero: ml/scripts/generar_parches.py

Orquesta lectura geoespacial, particionado train/validation/test y escritura YOLO-OBB. Relaciona ortomosaicos con capas vectoriales, detecta clases disponibles, reparte parcelas entre particiones, procesa parches en paralelo y genera la estructura de imágenes, etiquetas y reportes que consume Ultralytics.

ml/scripts/train.py (module)

Fichero: ml/scripts/train.py

Script de Entrenamiento para Modelos YOLOv11-OBB - vineGAPdetect Detección de

Marras con Oriented Bounding Boxes Uso básico: `python scripts/train.py -dataset-dir ./data/datasets/yolo_marras` Uso avanzado: `python scripts/train.py -dataset-dir ./data/datasets/yolo_marras -model models/pretrained/yolo11l-obbb.pt -epochs 100 -batch 16 -name experimento_v1`

ml/scripts/compare_models.py (module)

Fichero: `ml/scripts/compare_models.py`

Script de Comparación de Configuraciones - vineGAPdetect Ejecuta múltiples entrenamientos con diferentes tamaños de parche y modelos. Objetivo: Determinar el tamaño óptimo de parche para detección de marras. Comparación sistemática: - Modelos: nano (rápido) y large (preciso) - Tamaños de parche: 256, 320, 416, 512, 640 Uso: `python scripts/compare_models.py` Los resultados se guardan en `results/vineGAPdetect_Training/` con nombres descriptivos. Al finalizar, genera un reporte comparativo en `results/comparison_report.csv`

ml/scripts/evaluate_on_test.py (module)

Fichero: `ml/scripts/evaluate_on_test.py`

Script de Evaluación en Test Set - vineGAPdetect Valida todos los modelos `best.pt` en el conjunto de TEST (no validación). Esto proporciona métricas imparciales del rendimiento real de cada modelo. Uso: `python scripts/evaluate_on_test.py`

ml/scripts/explainability_cam.py (module)

Fichero: `ml/scripts/explainability_cam.py`

Script de Explicabilidad (XAI) para Modelos YOLOv11-OBb - vineGAPdetect. Genera visualizaciones offline con Eigen-CAM o Grad-CAM sobre imágenes de entrada para documentar el comportamiento del modelo fuera de la app web.

BaseCAM (class)

Fichero: `ml/scripts/yolo_cam/base_cam.py`

Base comun para generar mapas CAM sobre modelos YOLO usados en deteccion. Gestiona hooks de activacion y gradiente, escalado de mapas, suavizado y agregacion multicapa. Las implementaciones Eigen-CAM y Grad-CAM especializan el calculo de pesos o proyecciones sobre esta infraestructura comun.

EigenCAM (class)

Fichero: `ml/scripts/yolo_cam/eigen_cam.py`

Implementa Eigen-CAM usando la proyeccion principal de las activaciones.

GradCAM (class)

Fichero: `ml/scripts/yolo_cam/grad_cam.py`

Implementa Grad-CAM ponderando activaciones con gradientes medios por canal.

ActivationsAndGradients (class)

Fichero: `ml/scripts/yolo_cam/activations_and_gradients.py`

Extrae activaciones y registra gradientes de capas intermedias seleccionadas.